

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import matplotlib.cm as cm
from warnings import filterwarnings
filterwarnings('ignore')
```

```
In [4]: # Set professional color theme
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("viridis")

viridis_colors = cm.viridis(np.linspace(0,1,5))
primary_color = viridis_colors[0]
secondary_color = viridis_colors[1]
accent_color = viridis_colors[2]
danger_color = '#800000'
neutral_color = viridis_colors[4]
custom_palette = viridis_colors
```

```
In [6]: df = pd.read_csv('D:\DataCoSupplyChainDataset.csv', encoding = 'latin-1')
```

```
In [9]: # Exploratory Data Analysis(EDA)
```

```
In [11]: # Overview
print('rows, cols:',df.shape)
print('\ncolumns:')
print(df.columns.tolist())
print('\nNum duplicates:', df.duplicated().sum())
print('\nMissing values (top 20):')
print(df.isna().sum().sort_values(ascending = False).head(20))
```

```
rows, cols: (180519, 53)
```

```
columns:
```

```
['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit per order', 'Sales per customer', 'Delivery Status', 'Late_delivery_risk', 'Category Id', 'Category Name', 'Customer City', 'Customer Country', 'Customer Email', 'Customer Fname', 'Customer Id', 'Customer Lname', 'Customer Password', 'Customer Segment', 'Customer State', 'Customer Street', 'Customer Zipcode', 'Department Id', 'Department Name', 'Latitude', 'Longitude', 'Market', 'Order City', 'Order Country', 'Order Customer Id', 'order date (DateOrders)', 'Order Id', 'Order Item Cardprod Id', 'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id', 'Order Item Product Price', 'Order Item Profit Ratio', 'Order Item Quantity', 'Sales', 'Order Item Total', 'Order Profit Per Order', 'Order Region', 'Order State', 'Order Status', 'Order Zipcode', 'Product Card Id', 'Product Category Id', 'Product Description', 'Product Image', 'Product Name', 'Product Price', 'Product Status', 'shipping date (DateOrders)', 'Shipping Mode']
```

```
Num duplicates: 0
```

```
Missing values (top 20):
```

Product Description	180519
Order Zipcode	155679
Customer Lname	8
Customer Zipcode	3
Days for shipment (scheduled)	0
Sales per customer	0
Benefit per order	0
Delivery Status	0
Late_delivery_risk	0
Customer City	0
Customer Country	0
Category Id	0
Category Name	0
Customer Fname	0
Customer Email	0
Customer Password	0
Customer Id	0
Customer Segment	0
Customer State	0
Days for shipping (real)	0
dtype: int64	

```
In [22]: df.head()
```

```
Out[22]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>Type</th> <th>Days for shipping (real)</th> <th>Days for shipment (scheduled)</th> <th>Benefit per order</th> <th>Sales per customer</th> <th>Delivery Status</th> <th>Late_delivery_risk</th> <th>Category Id</th> <th>Category Name</th> <th>Customer City</th> <th>...</th> <th>Product Price</th> <th>Product Status</th> <th>shipping date (DateOrders)</th> <th>Shipping Mode</th> <th>Order Processing Time</th> <th>Delay</th> <th>Is_Delayed</th> <th>order_month</th> <th>order_day</th> <th>order_hour</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> </tbody>
```

DEBIT	3	4	91.250000	314.640015	Advance shipping	0	73	Sporting Goods	Caguas	...	327.75	0	2018-02-03 22:56:00	Standard Class
TRANSFER	5	4	-249.089996	311.359985	Late delivery	1	73	Sporting Goods	Caguas	...	327.75	0	2018-01-18 12:27:00	Standard Class
CASH	4	4	-247.779999	309.720001	Shipping on time	0	73	Sporting Goods	San Jose	...	327.75	0	2018-01-17 12:06:00	Standard Class
DEBIT	3	4	22.860001	304.809998	Advance shipping	0	73	Sporting Goods	Los Angeles	...	327.75	0	2018-01-16 11:45:00	Standard Class
PAYMENT	2	4	134.210007	298.250000	Advance shipping	0	73	Sporting Goods	Caguas	...	327.75	0	2018-01-15 11:24:00	Standard Class

<p>5 rows × 59 columns</p>



```
In [24]: # Data Cleaning
columns_to_drop = [
    'Product Description',
    'Product Image',
    'Customer Email',
    'Customer Password',
    'Customer Fname',
    'Customer Lname',
    'Customer Street',
    'Customer Zipcode',
    'Order Zipcode',
    'Longitude',
    'Latitude',
    'Order Item Cardprod Id',
    'Order Item Id',
    'Order Item Discount',
    'Order Item Discount Rate',
    'Order Item Product Price',
    'Order Item Quantity',
    'Order Item Total',
    'Category Id',
    'Department Id',
    'Order Id',
    'Order Customer Id',
    'Customer Id',
    'Product Card Id',
    'Product Category Id',
    'Benefit per order', # Identical to order profit per order
    'Product Status', # Having only one value
    'Customer City',
    'Order City',
    'Order Country',
    'Order State',
    'Customer State',
    'Market']

# Dropping columns that are either fully missing, redundant or have only one value (and thus no vari
```

```

df = df.drop(columns = columns_to_drop)

# Removing canceled orders since they are not relevant for delivery time analysis and may have diffe
df = df[df['Delivery Status'] != 'Shipping canceled']

# Standard date conversion
for c in ['order date (DateOrders)', 'shipping date (DateOrders)']:
    df[c] = pd.to_datetime(df[c], errors= 'coerce', dayfirst= False)

# After data cleaning, Lets check the overview again to see dataset has changed
print('rows, cols:', df.shape)
print('\nMissing values (top 5):')
print(df.isna().sum().sort_values(ascending = False).head(5))

```

rows, cols: (172765, 26)

Missing values (top 5):

Type	0
Days for shipping (real)	0
Days for shipment (scheduled)	0
Sales per customer	0
Delivery Status	0
dtype: int64	

In [25]: # Value counts for categorial columns with low cardinality

```

for col in df.columns:
    if df[col].nunique() < 10:
        print(f'\n{col},value counts:')
        print(df[col].value_counts())

```

Type,value counts:

Type

DEBIT	69295
TRANSFER	42129
PAYMENT	41725
CASH	19616

Name: count, dtype: int64

Days for shipping (real),value counts:

Days for shipping (real)

2	54205
6	27489
3	27478
4	27297
5	27003
0	4839
1	4454

Name: count, dtype: int64

Days for shipment (scheduled),value counts:

Days for shipment (scheduled)

4	103153
2	33806
1	26513
0	9293

Name: count, dtype: int64

Delivery Status,value counts:

Delivery Status

Late delivery	98977
Advance shipping	41592
Shipping on time	32196

Name: count, dtype: int64

Late_delivery_risk,value counts:

Late_delivery_risk

1	98977
0	73788

Name: count, dtype: int64

Customer Country,value counts:

Customer Country

EE. UU.	106425
Puerto Rico	66340

Name: count, dtype: int64

Customer Segment,value counts:

Customer Segment

Consumer	89420
Corporate	52528
Home Office	30817

Name: count, dtype: int64

Order Status,value counts:

Order Status

COMPLETE	59491
PENDING_PAYMENT	39832
PROCESSING	21902
PENDING	20227
CLOSED	19616
ON_HOLD	9804
PAYMENT_REVIEW	1893

Name: count, dtype: int64

Shipping Mode,value counts:

Shipping Mode

Standard Class	103153
Second Class	33806
First Class	26513
Same Day	9293

Name: count, dtype: int64

Order Processing Time,value counts:

Order Processing Time

2 54205

6 27489

3 27478

4 27297

5 27003

0 9293

Name: count, dtype: int64

Delay,value counts:

Delay

1 53503

0 36650

2 27551

-2 20873

-1 20719

3 6772

4 6697

Name: count, dtype: int64

Is_Delayed,value counts:

Is_Delayed

True 94523

False 78242

Name: count, dtype: int64

order_day,value counts:

order_day

Friday 24875

Sunday 24833

Saturday 24754

Thursday 24713

Tuesday 24585

Monday 24558

Wednesday 24447

Name: count, dtype: int64

```
In [26]: # Convert date columns to datetime
df['shipping date (DateOrders)'] = pd.to_datetime(
    df['shipping date (DateOrders)'],
    errors='coerce'
)

df['order date (DateOrders)'] = pd.to_datetime(
    df['order date (DateOrders)'],
    errors='coerce'
)

# Calculate order processing time
df['Order Processing Time'] = (
    df['shipping date (DateOrders)'] -
    df['order date (DateOrders)']
).dt.days

# Calculate delay
df['Delay'] = (
    df['Order Processing Time'] -
    df['Days for shipment (scheduled)']
)

# Delayed or not
df['Is_Delayed'] = df['Delay'] > 0

# Date features
df['order_month'] = df['order date (DateOrders)'].dt.month
df['order_day'] = df['order date (DateOrders)'].dt.day_name()
df['order_hour'] = df['order date (DateOrders)'].dt.hour

df.describe()
```

```
Out[26]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Days for shipping (real)</th> <th>Days for shipment (scheduled)</th> <th>Sales per customer</th> <th>Late_delivery_risk</th> <th>order date (DateOrders)</th> <th>Order Item Profit Ratio</th> <th>Sales</th> <th>Order Profit Per Order</th> <th>Product Price</th> <th>shipping date (DateOrders)</th> <th>Order Processing Time</th> <th>Delay</th> <th>order_month</th> <th>order_hour</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> <th>std</th> </tbody> <tbody></tbody>
```

count	mean	min	25%	50%	75%	max	std
172765.000000	172765.000000	172765.000000	172765.000000	172765	172765.000000	172765.000000	1
3.498596	2.933100	183.165948	0.572900	2016-06-12 15:25:39.457991	0.120801	203.828493	
0.000000	0.000000	7.490000	0.000000	2015-01-01 00:00:00	-2.750000	9.990000	
2.000000	2.000000	104.379997	0.000000	2015-09-21 18:01:00	0.080000	119.980003	
3.000000	4.000000	163.990005	1.000000	2016-06-11 08:11:00	0.270000	199.919998	
5.000000	4.000000	247.399994	1.000000	2017-02-28 21:08:00	0.360000	299.950012	
6.000000	4.000000	1939.989990	1.000000	2018-01-31 23:38:00	0.500000	1999.989990	
1.623446	1.373405	120.141871	0.494659	NaN	0.466610	132.392520	

```
In [27]: df['Is_Delayed'].value_counts()
```

```
Out[27]: Is_Delayed
True      94523
False     78242
Name: count, dtype: int64
```

```
In [28]: df.columns
```

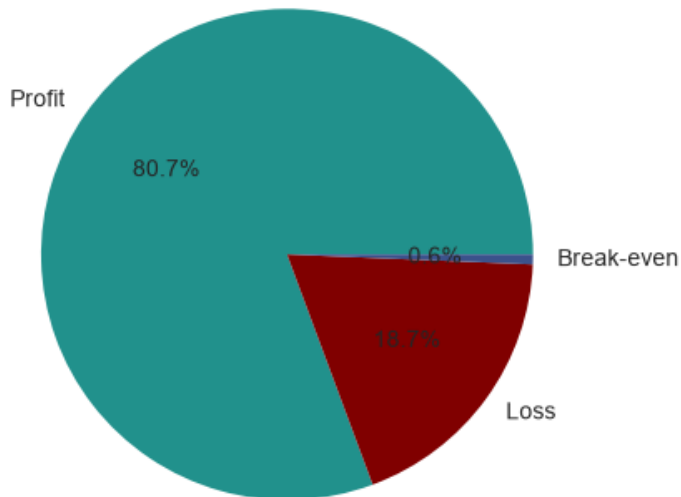
```
Out[28]: Index(['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)',
               'Sales per customer', 'Delivery Status', 'Late_delivery_risk',
               'Category Name', 'Customer Country', 'Customer Segment',
               'Department Name', 'order date (DateOrders)', 'Order Item Profit Ratio',
               'Sales', 'Order Profit Per Order', 'Order Region', 'Order Status',
               'Product Name', 'Product Price', 'shipping date (DateOrders)',
               'Shipping Mode', 'Order Processing Time', 'Delay', 'Is_Delayed',
               'order_month', 'order_day', 'order_hour'],
              dtype='str')
```

```
In [30]: # Profitability Flag based on order profit per order
df['Profitability Flag'] = np.where(df['Order Profit Per Order'] > 0, 'Profit', np.where(df['Order P
df['Profitability Flag'].value_counts()
```

```
Out[30]: Profitability Flag
Profit      139354
Loss        32295
Break-even    1116
Name: count, dtype: int64
```

```
In [31]: # Visulization of profitability distribution
profit_counts = df['Profitability Flag'].value_counts(normalize = True)*100
profit_counts.plot(kind='pie', autopct= '%1.1f%', colors = [accent_color, danger_color, secondary_c
plt.ylabel('')
plt.title('Profitability Distribution (%)')
plt.show()
```

Profitability Distribution (%)



```
In [35]: # =====
# BUSINESS KPI CALCULATIONS
# =====

def format_func(value):
    if value >= 1e6:
        return f'₹{value/1e6:.1f}M'
    elif value >= 1e3:
        return f'₹{value/1e3:.1f}K'
    else:
        return f'₹{value:.0f}'

# -----
# 1. Make sure date columns are datetime
# -----

df['shipping date (DateOrders)'] = pd.to_datetime(
    df['shipping date (DateOrders)'],
    errors='coerce'
)

df['order date (DateOrders)'] = pd.to_datetime(
    df['order date (DateOrders)'],
    errors='coerce'
)

# -----
# 2. Calculate Order Processing Time
# -----

df['Order Processing Time'] = (
    df['shipping date (DateOrders)']
    - df['order date (DateOrders)']
).dt.days

# -----
# 3. Calculate Delay
# -----

df['Delay'] = (
    df['Order Processing Time']
    - df['Days for shipment (scheduled)']
)
```

```

# -----
# 4. Delayed Flag
# -----

df['Is_Delayed'] = df['Delay'] > 0

# -----
# 5. Date Features
# -----

df['order_month'] = df['order date (DateOrders)'].dt.month

df['order_day'] = (
    df['order date (DateOrders)']
    .dt.day_name()
)

df['order_hour'] = (
    df['order date (DateOrders)']
    .dt.hour
)

# =====
# BUSINESS KPIs
# =====

# Orders with delay
delayed_df = df[df['Delay'] > 0]

metrics = {}

# Total Orders
metrics['Total Orders'] = len(df)

# Late Deliveries
metrics['Late Deliveries'] = len(delayed_df)

# 90th percentile delay
metrics['90% Delay (days)'] = (
    delayed_df['Delay'].quantile(0.90)
)

# On-time delivery percentage
metrics['On time Delivery %'] = (
    (1 - (float(metrics['Late Deliveries']) /
            metrics['Total Orders'])) * 100
)

# Late delivery percentage
metrics['Late Delivery %'] = (
    float(metrics['Late Deliveries']) /
    metrics['Total Orders'] * 100
)

# -----
# Profit calculations
# -----

# IMPORTANT:
# Correct column name is:
# 'Order Profit Per Order'

profit_col = 'Order Profit Per Order'

# Total Profit
metrics['Total Profit'] = format_func(
    df.loc[df[profit_col] > 0, profit_col].sum()
)

```

```

)

# Total Loss due to delays
metrics['Total Loss due to delays'] = format_func(
    df.loc[df['Delay'] > 0, profit_col].sum()
)

# =====
# PRINT BUSINESS KPIS
# =====

print("\n--- Business KPIS ---\n")

for k, v in metrics.items():

    if isinstance(v, float):
        print(f"{k}: {v:.2f}")

    else:
        print(f"{k}: {v}")

```

--- Business KPIS ---

```

Total Orders: 172765
Late Deliveries: 94523
90% Delay (days): 3.00
On time Delivery %: 45.29
Late Delivery %: 54.71
Total Profit: ₹7.5M
Total Loss due to delays: ₹2.1M

```

```

In [36]: # Profitability vs Delivery Time Analysis
profit_metrics = (
    df.groupby('Delay')['Order Profit Per Order']
    .agg(
        mean_profit='mean',
        total_profit='sum',
        order_count='count'
    )
    .reset_index()
)

```

In [37]: profit_metrics

```

Out[37]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe tthead th { text-align: right; } </style>
<thead>
  <th></th>
  <th>Delay</th>
  <th>mean_profit</th>
  <th>total_profit</th>
  <th>order_count</th>
</thead>
<tbody>
  <tr>
    <th>0</th>
    <th>1</th>
    <th>2</th>
    <th>3</th>
    <th>4</th>
    <th>5</th>
    <th>6</th>
  </tbody>
</tbody>

```

-2	23.360134	4.875961e+05	20873
-1	21.604769	4.476292e+05	20719
0	22.249118	8.154302e+05	36650
1	22.333227	1.194895e+06	53503
2	21.128491	5.821110e+05	27551
3	20.031412	1.356527e+05	6772
4	21.368783	1.431067e+05	6697

```

In [38]: delay_distribution = (
    df['Delay']
    .value_counts(normalize = True)
    .sort_index()*100
    ).reset_index()

```

In [39]: delay_distribution

```
Out[39]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe tthead th { text-align: right; } </style> <thead> <th></th> <th>Delay</th> <th>proportion</th> </thead> <tbody> <tr><th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> </tbody>
```

```
-2 12.081730
```

```
-1 11.992591
```

```
0 21.213788
```

```
1 30.968657
```

```
2 15.947096
```

```
3 3.919775
```

```
4 3.876364
```

```
In [40]: delay_distribution.columns = ['Delay_Days', 'Percentage']
```

```
print("\nProfit Metrics by Delay Day:")
display(profit_metrics.round(1))
```

```
print("\nDelay Distribution (%):")
display(delay_distribution)
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
```

```
# First subplot: Delay Distribution
```

```
sns.barplot(
    x='Delay_Days',
    y='Percentage',
    data=delay_distribution,
    color=accent_color,
    ax=ax1
)
```

```
ax1.set_title('Delay Distribution')
ax1.set_xlabel('Delay (days)')
ax1.set_ylabel('Percentage of Orders (%)')
```

```
# Percentage text on bars
```

```
for bar in ax1.patches:
    height = bar.get_height()
    ax1.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.5,
        f'{height:.1f}%',
        ha='center',
        va='bottom'
    )
```

```
# Second subplot: Profit Analysis by Delay Days
```

```
ax2.set_ylabel('Total Profit', color=primary_color)
```

```
ax2.bar(
    profit_metrics['Delay'],
    profit_metrics['total_profit'],
    color=primary_color,
    label='Total Profit'
)
```

```
ax2.tick_params(axis='y', labelcolor=primary_color)
```

```
ax3 = ax2.twinx()
```

```
ax3.set_xlabel('Delay Days')
ax3.set_ylabel('Mean Profit', color=accent_color)
```

```
ax3.plot(
    profit_metrics['Delay'],
```

```

profit_metrics['mean_profit'],
marker='o',
label='Mean Profit',
color=accent_color
)

ax3.tick_params(axis='y', labelcolor=accent_color)

plt.tight_layout()
plt.show()

```

Profit Metrics by Delay Day:

```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>
<thead>
<th></th>
<th>Delay</th>
<th>mean_profit</th>
<th>total_profit</th>
<th>order_count</th>
</thead>
<tbody>
<th>0</th>
<th>1</th>
<th>2</th>
<th>3</th>
<th>4</th>
<th>5</th>
<th>6</th>
</tbody>
</tbody>

```

```
-2 23.4 487596.1 20873
```

```
-1 21.6 447629.2 20719
```

```
0 22.2 815430.2 36650
```

```
1 22.3 1194894.7 53503
```

```
2 21.1 582111.0 27551
```

```
3 20.0 135652.7 6772
```

```
4 21.4 143106.7 6697
```

Delay Distribution (%):

```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>
<thead>
<th></th>
<th>Delay_Days</th>
<th>Percentage</th>
</thead>
<tbody>
<th>0</th>
<th>1</th>
<th>2</th>
<th>3</th>
<th>4</th>
<th>5</th>
<th>6</th>
</tbody>
</tbody>

```

```
-2 12.081730
```

```
-1 11.992591
```

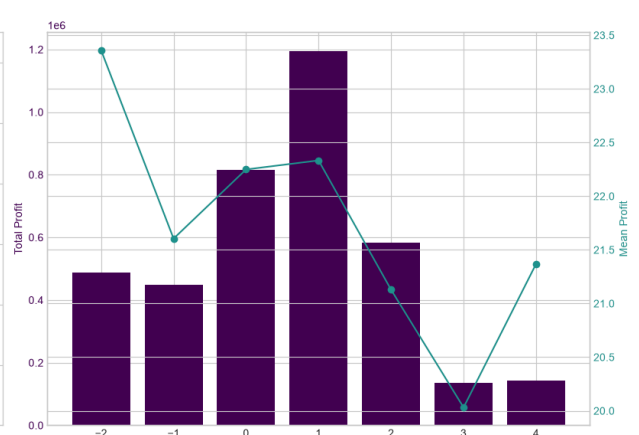
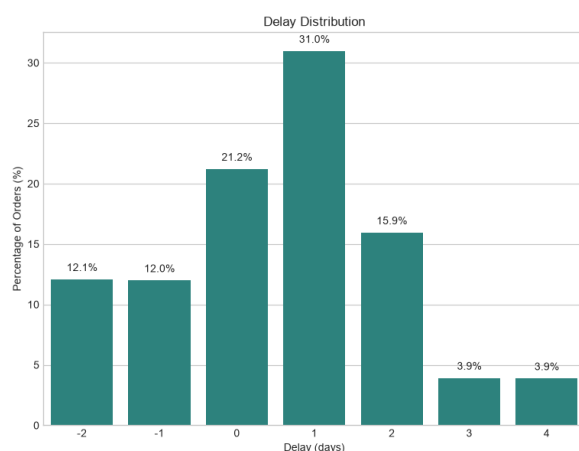
```
0 21.213788
```

```
1 30.968657
```

```
2 15.947096
```

```
3 3.919775
```

```
4 3.876364
```



```

In [41]: # Bottleneck Detection
def compute_delay_pct_by_category(category):
    cat_df = df.groupby(category).agg(
        total_orders=('Delay', 'count'),
        late_orders=('Is_Delayed', 'sum')
    ).reset_index()

```

```

cat_df['delay_pct'] = (
    cat_df['late_orders'] / cat_df['total_orders'] * 100
)

cat_df = cat_df.sort_values(
    'delay_pct',
    ascending=False
).head(10)

return cat_df

categories = [
    'Order Region',
    'Customer Segment',
    'Shipping Mode',
    'Order Status',
    'Type',
    'Department Name'
]

fig, axes = plt.subplots(
    2, 3,
    figsize=(16, 7),
    constrained_layout=True
)

axes = axes.flatten()

for ax, category in zip(axes, categories):

    cat_df = compute_delay_pct_by_category(category)

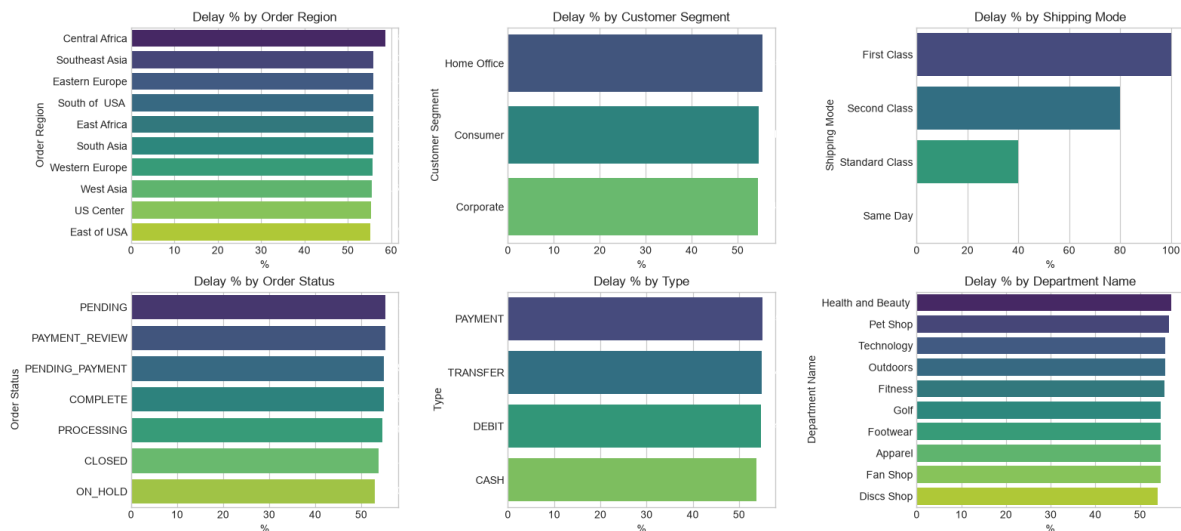
    sns.barplot(
        data=cat_df,
        x='delay_pct',
        y=category,
        ax=ax,
        palette='viridis'
    )

    ax.set_title(f'Delay % by {category}')
    ax.set_xlabel('%')
    ax.set_ylabel(category)

    for i, row in cat_df.reset_index().iterrows():
        ax.text(
            row['delay_pct'] + 1.5,
            i,
            f"{row['delay_pct']:.1f}%",
            va='center',
            fontsize=10,
            color='white'
        )

plt.show()

```



```
In [42]: # Root Cause Analysis
# Top drivers of late delivery by region

def top_drivers_for_region(region):

    df_region = df[df['Order Region'] == region].copy()

    drivers = [
        'Shipping Mode',
        'Customer Segment',
        'Department Name',
        'Type',
        'Order Status'
    ]

    all_factors = []

    for factor in drivers:

        temp = (
            df_region.groupby(factor)
            .agg(
                total_orders=('Delay', 'count'),
                late_orders=('Is_Delayed', 'sum'),
                avg_delay=('Delay', 'mean')
            )
            .reset_index()
        )

        temp['delay_pct'] = (
            temp['late_orders'] /
            temp['total_orders'] * 100
        )

        temp['Driver'] = factor

        temp['Factor_Level'] = (
            factor + " : " + temp[factor].astype(str)
        )

        all_factors.append(
            temp[
                [
                    'Driver',
                    'Factor_Level',
                    'delay_pct',
                    'avg_delay',
                    'total_orders'
                ]
            ]
        )
```

```

# Combine all drivers
final_df = pd.concat(all_factors)

# Top 10 drivers
top_factors = (
    final_df
    .sort_values('delay_pct', ascending=False)
    .head(10)
)

plt.figure(figsize=(12, 7))

bars = plt.barh(
    top_factors['Factor_Level'],
    top_factors['delay_pct']
)

plt.xlabel("Delay Percentage (%)")
plt.ylabel("Driver Factors")
plt.title(f"Top Drivers of Late Delivery in {region}")

plt.grid(
    True,
    linestyle='--',
    alpha=0.5
)

plt.gca().invert_yaxis()

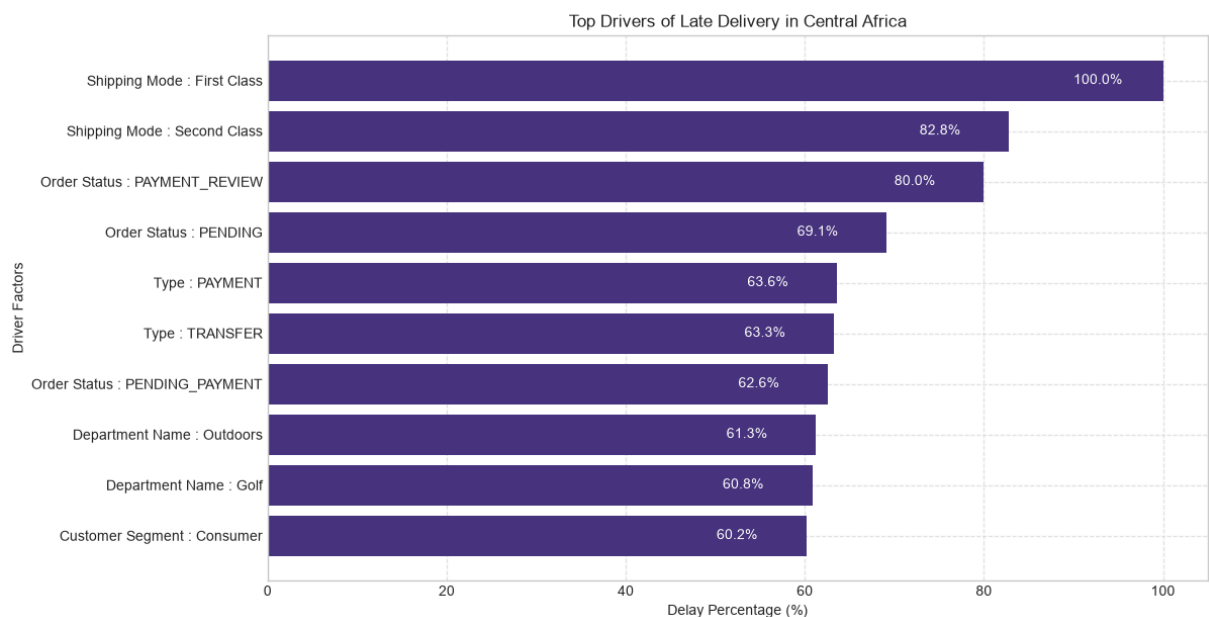
# Add percentage labels to bars
for bar in bars:
    width = bar.get_width()

    plt.text(
        width - 10,
        bar.get_y() + bar.get_height() / 2,
        f"{width:.1f}%",
        va='center',
        fontsize=10,
        color='white'
    )

plt.show()

# Run analysis for Central Africa
top_drivers_for_region('Central Africa')

```



```

In [43]: # Time Based Analysis
# Delay % by Month, Day of week , Hour
# Delay % by Month, Day of Week, Hour

delay_by_month = (
    df.groupby('order_month')['Is_Delayed']
      .mean()
      .reset_index()
)

delay_by_month['delay_pct'] = delay_by_month['Is_Delayed'] * 100

delay_by_day = (
    df.groupby('order_day')['Is_Delayed']
      .mean()
      .reset_index()
)

delay_by_day['delay_pct'] = delay_by_day['Is_Delayed'] * 100

delay_by_hour = (
    df.groupby('order_hour')['Is_Delayed']
      .mean()
      .reset_index()
)

delay_by_hour['delay_pct'] = delay_by_hour['Is_Delayed'] * 100

```

```

In [45]: fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(16, 6))

# =====
# Subplot 1: Delay % Trend Over Month
# =====

ax1.plot(
    delay_by_month['order_month'],
    delay_by_month['delay_pct'],
    marker='o',
    color=primary_color
)

ax1.set_xticks(range(1, 13))

ax1.set_xticklabels(
    ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
     'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
    rotation=45
)

ax1.set_xlabel("Month")
ax1.set_ylabel("Delay Percentage (%)")
ax1.set_title("Delay % Trend Over Month")

ax1.grid(
    True,
    linestyle=':',
    alpha=0.5
)

# Annotate top 3 highest months
top3_month = delay_by_month.nlargest(3, 'delay_pct')

for _, row in top3_month.iterrows():
    ax1.annotate(
        f"{row['delay_pct']:.1f}%",
        xy=(row['order_month'], row['delay_pct']),
        textcoords="offset points",
        xytext=(0, 10),
        ha='center',

```

```

        fontsize=10,
        color=danger_color
    )

# =====
# Subplot 2: Delay % by Day of Week
# =====

day_order = [
    'Monday',
    'Tuesday',
    'Wednesday',
    'Thursday',
    'Friday',
    'Saturday',
    'Sunday'
]

delay_by_day['order_day'] = pd.Categorical(
    delay_by_day['order_day'],
    categories=day_order,
    ordered=True
)

delay_by_day = delay_by_day.sort_values('order_day')

ax2.bar(
    delay_by_day['order_day'],
    delay_by_day['delay_pct'],
    color=primary_color
)

ax2.set_xticklabels(
    delay_by_day['order_day'],
    rotation=45
)

ax2.set_xlabel("Day of Week")
ax2.set_ylabel("Delay Percentage (%)")
ax2.set_title("Delay % by Day of Week")

ax2.grid(
    True,
    linestyle=':',
    alpha=0.5
)

# Annotate top 3 highest bars
top3_day = delay_by_day.nlargest(3, 'delay_pct')

for _, row in top3_day.iterrows():
    height = row['delay_pct']

    ax2.text(
        row['order_day'],
        height + 0.5,
        f"{height:.1f}%",
        ha='center',
        va='bottom',
        fontsize=10,
        color=danger_color
    )

# =====
# Subplot 3: Delay % by Hour
# =====

ax3.plot(
    delay_by_hour['order_hour'],
    delay_by_hour['delay_pct'],

```

```

marker='o',
color=primary_color
)

ax3.set_xlabel("Hour of Day")
ax3.set_ylabel("Delay Percentage (%)")
ax3.set_title("Delay % by Hour")

ax3.grid(
    True,
    linestyle=':',
    alpha=0.5
)

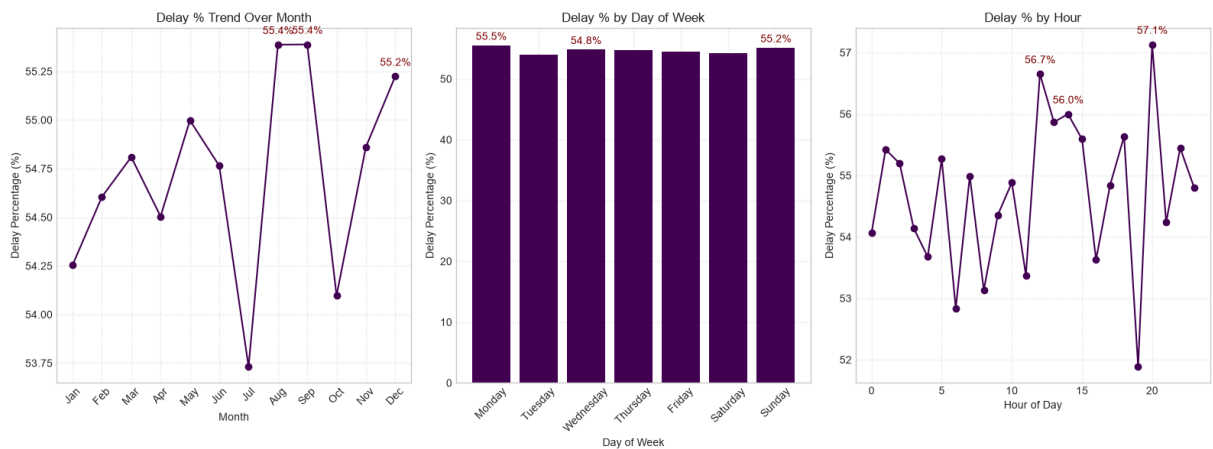
# Annotate top 3 highest hours
top3_hour = delay_by_hour.nlargest(3, 'delay_pct')

for _, row in top3_hour.iterrows():
    ax3.annotate(
        f"{row['delay_pct']:.1f}%",
        xy=(row['order_hour'], row['delay_pct']),
        textcoords="offset points",
        xytext=(0, 10),
        ha='center',
        fontsize=10,
        color=danger_color
    )

# =====
# Final Layout
# =====

plt.tight_layout()
plt.show()

```



In []: